

High-Performance and Power-Efficient Emulation of Matrix Multiplication using INT8 Matrix Engines

Yuki Uchino*

RIKEN Center for Computational
Science (R-CCS)
Kobe, Hyogo, Japan
yuki.uchino.fe@riken.jp

Katsuhisa Ozaki*

Shibaura Institute of Technology
Saitama, Saitama, Japan
ozaki@sic.shibaura-it.ac.jp

Toshiyuki Imamura*

RIKEN Center for Computational
Science (R-CCS)
Kobe, Hyogo, Japan
imamura.toshiyuki@riken.jp

Abstract

Recent architectures integrate high-performance and power-efficient matrix engines. These engines demonstrate remarkable performance in low-precision matrix multiplication, which is crucial in deep learning. Several techniques have been proposed to emulate single- and double-precision general matrix-matrix multiplication (SGEMM and DGEMM, respectively) by leveraging such low-precision matrix engines. In this study, we present emulation methods that significantly outperforms conventional approaches. On a GH200 Grace Hopper Superchip, the proposed DGEMM emulation achieves a 1.4 $\times$ speedup and a 43% improvement in power efficiency compared to native DGEMM for sufficiently large problems. The proposed SGEMM emulation achieves a 3.0 $\times$ speedup and a 154% improvement in power efficiency compared to native SGEMM for sufficiently large problems. Furthermore, compared to conventional emulation methods, the proposed emulation achieves more than 2 $\times$ higher performance and superior power efficiency.

CCS Concepts

• Mathematics of computing $\rightarrow$ Mathematical software performance; • Computer systems organization $\rightarrow$ Multicore architectures.

Keywords

Matrix Multiplication, Emulation, High-Performance Computing, Mixed-Precision Computing, Power Efficient

ACM Reference Format:

Yuki Uchino, Katsuhisa Ozaki, and Toshiyuki Imamura. 2025. High-Performance and Power-Efficient Emulation of Matrix Multiplication using INT8 Matrix Engines. In *Workshops of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC Workshops '25)*, November 16–21, 2025, St Louis, MO, USA. ACM, New York, NY, USA, 8 pages. <https://doi.org/10.1145/3731599.3767539>

1 Introduction

This paper proposes high-performance methods for emulating single- and double-precision general matrix-matrix multiplication

(SGEMM and DGEMM, respectively) on modern architectures. Recent architectures include dedicated low-precision matrix engines, such as NVIDIA Tensor Cores [7], AMD Matrix Cores [1], Intel AI Boost Neural Processing Units (NPU) [16], and Google Cloud Tensor Processing Units (TPU) [4], designed to accelerate the matrix operations required for deep learning. Figure 1 shows TFLOPS and TOPS of AMD and NVIDIA GPUs. The performance of low-precision engines has significantly improved with each new generation. However, the performance increases of FP64 and FP32 are modest compared to lower precision types. This trend poses a challenge for the high-performance computing field, where high-precision calculations remain essential. To overcome these challenges, various numerical mixed-precision algorithms utilizing low-precision matrix engines have been proposed.

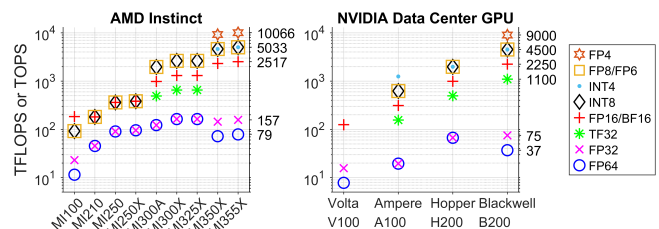


Figure 1: TFLOPS and TOPS of AMD and NVIDIA GPUs for dense data

The present study focuses on the emulation of matrix multiplication. Note that matrix multiplication involving tall-and-skinny or small-scale matrices is not considered in this paper, as such cases fail to fully utilize the computational capabilities of matrix engines and tend to expose performance bottlenecks in the emulation, resulting in memory-bound behavior. Recently, we proposed Ozaki scheme II, a highly accurate matrix multiplication emulation based on the Chinese Remainder Theorem (CRT) [15]. In the present study, we provide DGEMM and SGEMM emulation based on Ozaki scheme II using INT8 matrix engines, together with elaborately designed implementation techniques. The code is available from [18]. Numerical experiment results show that our emulation is faster than native DGEMM and SGEMM for sufficiently large matrices.

The remainder of this paper is organized as follows. Section 2 introduces emulation methods proposed in the previous studies. Section 3 gives an overview of Ozaki scheme II. Section 4 presents the proposed efficient implementation for emulating DGEMM and SGEMM. Section 5 shows the numerical results of the proposed emulation. Section 6 presents the concluding remarks.

*All authors contributed equally to this research.



This work is licensed under a Creative Commons Attribution 4.0 International License. *SC Workshops '25, St Louis, MO, USA*

© 2025 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-1871-7/25/11
<https://doi.org/10.1145/3731599.3767539>

2 Related Work

Matrix engines can perform mixed-precision computations, accumulating the results of matrix multiplication with higher-precision than that of the input matrices. Leveraging this property, several methods for emulating higher-precision matrix multiplication have been developed. Emulation techniques utilizing such matrix engines are emerging as an essential foundation of non-AI computations. In the numerical experiments presented in Section 5, we compare our emulation with these existing emulation methods.

Let $\mathbb{F}_b$ be the set of b -bit binary floating-point numbers for $b \in \mathbb{N}$. For example, $\mathbb{F}_{32}$ and $\mathbb{F}_{64}$ are the sets of FP32 and FP64, respectively. Let $\mathbb{Z}_b$ and $\mathbb{U}_b$ be the sets of b -bit signed and unsigned integers, respectively, for $b \in \mathbb{N}$. cuMpSGEMM emulates SGEMM using FP16 and TF32 Tensor Cores [8, 10, 12]. It decomposes input matrices $A \in \mathbb{F}_{32}^{m \times k}$ and $B \in \mathbb{F}_{32}^{k \times n}$ into $A \approx A_1 + s^{-1}A_2$ and $B \approx B_1 + s^{-1}B_2$, respectively, where A_i and B_i are represented in either FP16 or TF32 format for $i \in \{1, 2\}$, and $s := 2^{-11}$ for FP16 and $s := 1$ for TF32. After the decomposition, AB is approximated as $AB \approx A_1B_1 + s^{-1}(A_1B_2 + A_2B_1)$. cuMpSGEMM offers several computing modes. Among them, this study uses FP16TCEC_SCALING mode, which can emulate SGEMM using FP16 Tensor Cores without accuracy loss and with higher performance than SGEMM. Two FP16 values cannot fully capture the mantissa of a FP32 value; therefore, error correction is applied to restore the lost mantissa precision. As a related approach, NVIDIA cuBLAS 12.9 supports the BF16x9 algorithm for emulating SGEMM on Blackwell GPUs [5]. It decomposes A and B into BF16 matrices as $A = A_1 + 2^{-8}A_2 + 2^{-16}A_3$ and $B = B_1 + 2^{-8}B_2 + 2^{-16}B_3$. Then AB is computed based on $AB = \sum_{i=1}^3 \sum_{j=1}^3 2^{-8(i+j-2)} A_i B_j$. A similar approach using fused multiply-add (FMA) operations was proposed in [6].

The Ozaki scheme (Ozaki scheme I), a highly accurate matrix multiplication based on an error-free transformation of a matrix product [13, 14], can contribute to the design of emulation for higher-precision matrix multiplication using low-precision matrix engines. For $in, out \in \mathbb{N}$ and a user-specified $d \in \mathbb{N}$, Ozaki scheme I splits two input matrices $A \in \mathbb{F}_{in}^{m \times k}$ and $B \in \mathbb{F}_{in}^{k \times n}$ into $A \approx \sum_{i=1}^d A_i$ and $B \approx \sum_{i=1}^d B_i$, respectively, where $A_i \in \mathbb{F}_{out}^{m \times k}$, $B_i \in \mathbb{F}_{out}^{k \times n}$, and no rounding error occurs in computing $A_i B_j$ for $1 \leq i, j \leq d$. Then, AB is approximated as $AB \approx \sum_{i+j \leq d+1} A_i B_j$, involving $d(d+1)/2$ matrix multiplications. By appropriately extracting A_i and B_j and applying diagonal scaling to them, $A_i B_j$ can be computed in low precision. In this paper, we assume that INT8 matrix engines take INT8 as an input and accumulate in INT32. DGEMM emulation based on Ozaki scheme I using INT8 matrix engines is proposed in [9, 11, 17, 19].

3 Ozaki Scheme II

Hereafter, for any $X \in \mathbb{Z}^{m \times n}$ and $p \in \mathbb{N}$, we write

$$\text{mod}(X, p) := X - p\lfloor X/p \rfloor, \quad \text{rmod}(X, p) := X - p \cdot \text{round}(X/p),$$

where $\text{round}(\cdot)$ rounds an input matrix to the nearest integers. The methods introduced in Section 2 decompose A and B into multiple-component forms by splitting significands and transform AB into a sum of the products of the corresponding components. In contrast, Ozaki Scheme II is based on the CRT (Theorem 1); it employs a technique that avoids splitting the input matrices.

The theorem implies that one of the solutions of a sequence of congruence equations (1) is $x = \text{rmod}(\sum_{i=1}^N \mathcal{P}/p_i \cdot q_i \cdot y_i, \mathcal{P})$. If $\mathcal{P}$ is sufficiently large for $|x|$, the solution becomes unique. Ozaki scheme II utilizes this property to emulate matrix multiplication.

THEOREM 1 (CHINESE REMAINDER THEOREM). *Let $x \in \mathbb{Z}$. Suppose that $p_1, \dots, p_N \in \mathbb{N}_{\geq 2}$ are pairwise coprime integers and $\mathcal{P} := \prod_{1 \leq i \leq N} p_i$. For $i = 1, \dots, N$, define $q_i \in \mathbb{N}$ as modular multiplicative inverses of $\mathcal{P}/p_i$ (i.e., $\mathcal{P}/p_i \cdot q_i \equiv 1 \pmod{p_i}$). Let $y_i \in \mathbb{Z}$ for $i = 1, \dots, N$ be such that*

$$\begin{cases} x \equiv y_1 \pmod{p_1}, \\ \vdots \\ x \equiv y_N \pmod{p_N}. \end{cases} \quad (1)$$

Then, it holds that

$$x \equiv \sum_{i=1}^N \frac{\mathcal{P}}{p_i} q_i y_i \pmod{\mathcal{P}}. \quad (2)$$

Assume that $N \in \mathbb{N}$ is specified by a user. Ozaki scheme II for computing $C \approx AB$ is constructed as follows:

Step 1. Determine pairwise coprime integers $p_1, \dots, p_N \in \mathbb{N}_{\geq 2}$ and calculate $\mathcal{P} := \prod_{1 \leq i \leq N} p_i \in \mathbb{N}$ and modular multiplicative inverses $q_1, \dots, q_N \in \mathbb{N}$ of $\mathcal{P}/p_i$. $q_i < p_i$ holds.

Step 2. Apply diagonal scaling and truncation to convert $A \in \mathbb{F}^{m \times k}$ and $B \in \mathbb{F}^{k \times n}$ to

$$A' := \text{trunc}(\text{diag}(\mu) \cdot A) \in \mathbb{Z}^{m \times k}, \quad \mu \in \mathbb{F}^m,$$

$$B' := \text{trunc}(B \cdot \text{diag}(\nu)) \in \mathbb{Z}^{k \times n}, \quad \nu \in \mathbb{F}^n,$$

respectively, where $\text{trunc}(\cdot)$ rounds an input matrix toward zero and all elements of μ and ν are powers of two chosen to satisfy

$$2 \sum_{h=1}^k |a'_{ih}| |b'_{hj}| < \mathcal{P} \quad \forall i, j. \quad (3)$$

Step 3. Compute $C'' := A'B'$ via the CRT as

$$C' := \sum_{i=1}^N \frac{\mathcal{P}}{p_i} q_i \cdot \text{rmod}(A', p_i) \cdot \text{rmod}(B', p_i), \quad (4)$$

$$C'' := \text{rmod}(C', \mathcal{P}). \quad (5)$$

Step 4. Inversely scale C'' as

$$C := \text{diag}(\mu^{-1}) \cdot C'' \cdot \text{diag}(\nu^{-1}).$$

p_i are chosen so that no error occurs in $\text{rmod}(A', p_i) \cdot \text{rmod}(B', p_i)$ in (4). Steps 3 performs integer matrix multiplication $A'B'$ via the CRT; $\text{rmod}(A', p_i) \cdot \text{rmod}(B', p_i)$ in (4) and C'' in (5) correspond to y_i in (1) and x in (2), respectively. As discussed in [15], (3) yields the uniqueness of C'' . In Step 2, A and B are transformed into A' and B' , respectively, to ensure the uniqueness of the CRT result. Increasing N enlarges $\mathcal{P}$ in (3), which reduces the truncation error in Step 2. Therefore, the accuracy of $C \approx AB$ depends on the number of moduli. In the following sections, we provide algorithms and coding techniques of Ozaki scheme II for emulating DGEMM and SGEMM using INT8 matrix engines.

4 DGEMM and SGEMM Emulation

In DGEMM and SGEMM emulation, the matrix multiplication in (4) and the other computations are performed using INT8 matrix engines and high-precision operations, respectively. INT8 matrix engines are overwhelmingly faster than high-precision computations. Therefore, even if the computational cost of high-precision operations is less than that of INT8 operations, their execution time remains significant and cannot be ignored. Consequently, if Steps 2 to 4 in Section 3 are implemented in a straightforward manner, they become bottlenecks, hindering performance. Hence, it is crucial to skillfully apply techniques such as table lookups and efficient error-free computations to avoid division and slow high-precision computations. Taking these considerations into account, Algorithm 1 presents the proposed emulation. This section describes the constants and implementation details used in Algorithm 1.

Algorithm 1 DGEMM and SGEMM emulation via INT8

Input: $A \in \mathbb{F}_b^{m \times k}$, $B \in \mathbb{F}_b^{k \times n}$ for $b \in \{32, 64\}$, $N \in \mathbb{N}_{\geq 2}$

- 1: Determine scale vectors $\mu \in \mathbb{F}_b^m$ and $\nu \in \mathbb{F}_b^n$
- 2: $A' \leftarrow \text{trunc}(\text{diag}(\mu) \cdot A) \in \mathbb{F}_b^{m \times k} \cap \mathbb{Z}^{m \times k}$
- 3: $B' \leftarrow \text{trunc}(B \cdot \text{diag}(\nu)) \in \mathbb{F}_b^{k \times n} \cap \mathbb{Z}^{k \times n}$
- 4: $A'_i \leftarrow \text{rmod}(A', p_i) \in \mathbb{Z}_8^{m \times k}$ for $1 \leq i \leq N$
- 5: $B'_i \leftarrow \text{rmod}(B', p_i) \in \mathbb{Z}_8^{k \times n}$ for $1 \leq i \leq N$
- 6: $C'_i \leftarrow A'_i B'_i \in \mathbb{Z}_{32}^{m \times n}$ for $1 \leq i \leq N$ using INT8 matrix engines
- 7: $U_i \leftarrow \text{mod}(C'_i, p_i) \in \mathbb{U}_8^{m \times n}$ for $1 \leq i \leq N$
- 8: $C'^{(1)} \leftarrow \sum_{i=1}^N s_{i1} U_i \in \mathbb{F}_{64}^{m \times n}$
- 9: $C'^{(2)} \leftarrow \sum_{i=1}^N s_{i2} U_i \in \mathbb{F}_{64}^{m \times n}$
- 10: $Q \leftarrow \text{round}(\mathcal{P}_{\text{inv}} \cdot C'^{(1)}) \in \mathbb{F}_{64}^{m \times n} \cap \mathbb{Z}^{m \times n}$
- 11: $C'' \leftarrow ((C'^{(1)} - \mathcal{P}_1 \cdot Q) + C'^{(2)}) - \mathcal{P}_2 \cdot Q \in \mathbb{F}_{64}^{m \times n}$ using FMA
- 12: $C \leftarrow \text{diag}(\mu^{-1}) \cdot C'' \cdot \text{diag}(\nu^{-1}) \in \mathbb{F}_{64}^{m \times n}$

4.1 Determination of Constants

When using INT8 matrix engines, the pairwise coprime integers $p_1, \dots, p_N \in \mathbb{N}_{\geq 2}$ can be fixed as

$$p_s \in \{256, 255, 253, 251, \dots, 41, 37, 29\} \setminus \{p_t \mid t \neq s, 1 \leq t \leq N\}$$

to satisfy $-128 \leq (A'_s)_{ij} \leq 127$ and $-128 \leq (B'_s)_{ij} \leq 127$. The values $\text{rmod}(A', 256)$ and $\text{rmod}(B', 256)$ can be 128, but casting to INT8 wraps them around to -128 . This is not an issue because $128 \equiv -128 \pmod{256}$. In practice, $N \leq 20$ is sufficient for DGEMM emulation, while $N \leq 10$ is sufficient for SGEMM emulation. To prevent the table size from becoming excessive, we assume $N \leq 20$ in the implementation.

We define $\text{double}(\cdot)$ and $\text{single}(\cdot)$ to denote rounding to nearest double- and single-precision floating-point numbers, respectively. Since p_i is fixed, both $\mathcal{P}$ and q_i are known in advance. Therefore, the implementation constructs a table of p_i , $\mathcal{P}$, and $\mathcal{P}/p_i \cdot q_i$ for each N . In DGEMM emulation, accumulation in Step 3 requires arithmetic with precision higher than double precision; thus, $\mathcal{P}$ is stored as a double-double number:

$$\mathcal{P}_1 := \text{double}(\mathcal{P}) \in \mathbb{F}_{64}, \quad \mathcal{P}_2 := \text{double}(\mathcal{P} - \mathcal{P}_1) \in \mathbb{F}_{64},$$

and $\mathcal{P}/p_i \cdot q_i$ is approximated using two double-precision floating-point numbers $s_{i1}, s_{i2} \in \mathbb{F}_{64}$ in the form

$$\frac{\mathcal{P}}{p_i} \cdot q_i \approx s_{i1} + s_{i2}, \quad (6)$$

where this is not a double-double number. As discussed later (see Section 4.3), to reduce the computational cost of the accumulation, s_{i1} retains only the upper β_i bits of $\mathcal{P}/p_i \cdot q_i$ for

$$\beta_i := 53 - 8 - \lceil \log_2 N \rceil + \left\lceil \log_2 \frac{\mathcal{P}}{p_i} q_i \right\rceil - \left\lceil \log_2 \max_{1 \leq j \leq N} \frac{\mathcal{P}}{p_j} q_j \right\rceil,$$

while s_{i2} holds the upper 53 bits of the remainder. Figure 2 shows an image of s_{i1} and s_{i2} .

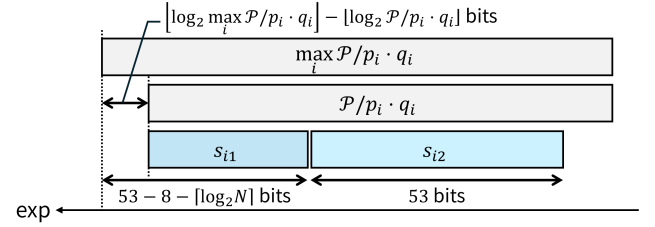


Figure 2: Image of s_{i1} and s_{i2}

In SGEMM emulation, we prepare $\mathcal{P}$ and $\mathcal{P}/p_i \cdot q_i$ as double-precision floating-point numbers:

$$\mathcal{P}_1 := \text{double}(\mathcal{P}) \in \mathbb{F}_{64}, \quad s_{i1} := \text{double}\left(\frac{\mathcal{P}}{p_i} \cdot q_i\right) \in \mathbb{F}_{64},$$

and we set $\mathcal{P}_2 = s_{i2} = 0$. In addition, for later use,

$$\text{pinv}_i^{(64)} := \text{double}(1/p_i) \in \mathbb{F}_{64}, \quad \text{pinv}_i^{(32)} := \text{single}(1/p_i) \in \mathbb{F}_{32},$$

$$\text{pinv}'_i := \lfloor 2^{32}/p_i - 1 \rfloor \in \mathbb{Z}_{32}, \quad \mathcal{P}_{\text{inv}} := \text{double}(\mathcal{P}^{-1}) \in \mathbb{F}_{64},$$

$$\mathcal{P}'_{\text{fast}} := \text{single}(\log_2(\mathcal{P} - 1)/2 - 1.5) \in \mathbb{F}_{32},$$

$$\mathcal{P}'_{\text{accu}} := \text{single}(\log_2(\mathcal{P} - 1)/2 - 0.5) \in \mathbb{F}_{32}$$

are also prepared.

4.2 Conversion to INT8

We provide two computing modes: fast mode and accurate mode. In fast mode, the scale vectors $\mu \in \mathbb{F}_b^m$ and $\nu \in \mathbb{F}_b^n$ are determined using the Cauchy–Schwarz inequality as

$$2 \sum_{h=1}^k |a'_{ih}| |b'_{hj}| \leq 2\mu_i \|a_{i,:}\|_2 \|b_{:,j}\|_2 \nu_j < \mathcal{P}. \quad (7)$$

To satisfy this, μ and ν are set as

$$\mu_i := 2^{\left\lfloor \mathcal{P}'_{\text{fast}} - \max\left(1, 0.51 \cdot \log_2 \sum_{j=1}^k a_{ij}^2\right) \right\rfloor - \left\lfloor \log_2 \max_{1 \leq h \leq k} |a_{ih}| \right\rfloor},$$

$$\nu_j := 2^{\left\lfloor \mathcal{P}'_{\text{fast}} - \max\left(1, 0.51 \cdot \log_2 \sum_{i=1}^k b_{ij}^2\right) \right\rfloor - \left\lfloor \log_2 \max_{1 \leq h \leq k} |b_{hj}| \right\rfloor},$$

where $\sum_{j=1}^k a_{ij}^2$ and $\sum_{i=1}^k b_{ij}^2$ should be computed using floating-point arithmetic in round-up mode. In accurate mode, the scale vectors $\mu' \in \mathbb{F}_b^m$ and $\nu' \in \mathbb{F}_b^n$ are determined as

$$\mu'_i := 2^{5 - \left\lfloor \log_2 \max_{1 \leq h \leq k} |a_{ih}| \right\rfloor}, \quad \nu'_j := 2^{5 - \left\lfloor \log_2 \max_{1 \leq h \leq k} |b_{hj}| \right\rfloor},$$

and then A and B are converted to INT8 matrices $\bar{A} \in \mathbb{Z}^{m \times k}$ and $\bar{B} \in \mathbb{Z}^{k \times n}$, respectively, as follows:

$$\bar{a}_{ij} := \lceil \mu'_i \cdot |a_{ij}| \rceil \leq 2^7 - 1, \quad \bar{b}_{ij} := \lceil |b_{ij}| \cdot v'_j \rceil \leq 2^7 - 1.$$

Then, the scale vectors μ and v can be determined to satisfy

$$\begin{aligned} 2 \sum_{h=1}^k |a'_{ih}| |b'_{hj}| &\leq 2\mu_i \left(\sum_{h=1}^k |a_{ih}| |b_{hj}| \right) v_j \\ &\leq 2\mu_i \mu_i'^{-1} \left(\bar{A} \cdot \bar{B} \right)_{ij} v_j'^{-1} v_j < \mathcal{P}. \end{aligned}$$

This mode computes $\bar{C} := \bar{A} \cdot \bar{B}$ using INT8 matrix engines. Then μ and v are set as

$$\begin{aligned} \mu_i &:= \mu'_i \cdot 2^{\lfloor \mathcal{P}'_{\text{accu}} - 0.51 \cdot \log_2(\max_{1 \leq h \leq n} \bar{c}_{ih}) \rfloor}, \\ v_j &:= v'_j \cdot 2^{\lfloor \mathcal{P}'_{\text{accu}} - 0.51 \cdot \log_2(\max_{1 \leq h \leq m} \bar{c}_{hj}) \rfloor}. \end{aligned}$$

In terms of computation speed, fast mode is quicker than accurate mode because accurate mode requires matrix multiplication $A \cdot B$. However, when the accuracy of results is required, accurate mode should be used. It outputs more accurate results than those of fast mode because the Cauchy–Schwarz inequality overestimates the upper bound of $\sum_{h=1}^k |a_{ih}| |b_{hj}|$ in (7), whereas accurate mode estimates the upper bound by direct matrix multiplication.

After the scale values have been determined, A and B are converted to INT8 matrices $A'_1, \dots, A'_N \in \mathbb{Z}_8^{m \times k}$ and $B'_1, \dots, B'_N \in \mathbb{Z}_8^{k \times n}$, respectively, as in the lines 4 and 5 of Algorithm 1. Since the built-in modular arithmetic function $\text{fmod}(\cdot)$ is not efficient, we implement $y := \text{rmod}(x, p_i)$ for $x \in \mathbb{F}_b$ for $b \in \{32, 64\}$ as follows to achieve better performance:

- (1) $y \leftarrow \text{single}(\text{fma}(\text{round}(x \cdot \text{pinv}_i^{(b)}), -p_i, x))$,
- (2) if $N \geq N_1$, $y \leftarrow \text{fma}(\text{round}(y \cdot \text{pinv}_i^{(32)}), -p_i, y)$,
- (3) if $N \geq N_2$, $y \leftarrow \text{fma}(\text{round}(y \cdot \text{pinv}_i^{(32)}), -p_i, y)$,

where the function $\text{fma}(x, y, z)$ calculates $xy + z$ using FMA and $(N_1, N_2) = (13, 19)$ and $(N_1, N_2) = (5, 11)$ for $b = 64$ and $b = 32$, respectively. This $\text{rmod}(x, p_i)$ works well for $N \leq 18$ when $b = 32$ and for $N \leq 20$ when $b = 64$.

4.3 Accumulation

For $i = 1, \dots, N$, INT32 matrices $C'_i := A'_i \cdot B'_i$ are calculated using INT8 matrix engines. Performing FP64 accumulation in each matrix multiplication step to compute $C' := \sum_{i=1}^N \mathcal{P}/p_i \cdot q_i \cdot C'_i$ creates a performance bottleneck. Thus, we convert C'_i to UINT8 matrices U_i as in the line 7 of Algorithm 1, then accumulate all U_i in a single kernel invocation in the next stage. We use $\text{mod}(C'_i, p_i)$ instead of $\text{rmod}(C'_i, p_i)$ for better performance. Because integer modulo operation % is relatively slow, we implemented $\text{mod}(x, p_i)$ for $x \in \mathbb{Z}_{32}$ as follows:

- (1) $y \leftarrow x - _\text{mulhi}(x, \text{pinv}_i') \cdot p_i$,
- (2) $y \leftarrow y - (y \geq p_i) \cdot p_i + (y < 0) \cdot p_i$,

where $_\text{mulhi}$ calculates the most significant 32 bits of the product.

We assume that $k < 2^{17}$. For $k < 2^{17}$, $A'_i \cdot B'_i$ can be calculated without error because all elements of the product of $A'_i \cdot B'_i$ are less than 2^{31} . For $k = 2^{17}$, $p_1 = 256$ implies $(A'_1 \cdot B'_1)_{ij} \leq 2^{31}$. Even when $(A'_1 \cdot B'_1)_{ij} = 2^{31}$, wraparound occurs and we obtain $(C'_1)_{ij} = -2^{31}$. Since $\text{rmod}(-2^{31}, 256) = \text{rmod}(2^{31}, 256) = 0$, no error is introduced

for computing $(U_1)_{ij}$. If we have case $k > 2^{17}$, it is possible to apply block matrix multiplication.

Using U_i , C' in (4) can be represented as $C' := \sum_{i=1}^N \mathcal{P}/p_i \cdot q_i \cdot U_i$. We compute $C' \approx C'^{(1)} + C'^{(2)}$ as

$$C'^{(1)} \leftarrow \sum_{i=1}^N s_{i1} U_i, \quad C'^{(2)} \leftarrow \sum_{i=1}^N s_{i2} U_i,$$

where s_{i1} and s_{i2} are defined as in (6) and no rounding error occurs in $\sum_{i=1}^N s_{i1} U_i$ in FP64. While direct accumulation of $C'_i \in \mathbb{Z}_{32}^{m \times n}$ lacks sufficient accuracy, using $U_i \in \mathbb{U}_8^{m \times n}$ enables sufficiently accurate summation. Then, we compute

$$Q \leftarrow \text{round}(\mathcal{P}_{\text{inv}} \cdot C'^{(1)}) = \text{round}(C' / \mathcal{P}) \quad (8)$$

and C'' in (5) as

$$C'' \leftarrow \text{fma}(-\mathcal{P}_2, Q, \text{fma}(-\mathcal{P}_1, Q, C'^{(1)}) + C'^{(2)}) \approx C' - \mathcal{P}Q.$$

A rigorous error analysis of the proposed method, which will also establish the validity of (8), will be pursued as future work.

5 Numerical Results

Numerical experiments in this paper were conducted on an NVIDIA GeForce RTX 5080 Blackwell GPU with an AMD Ryzen 9 7950X CPU, an NVIDIA Ampere A100 SXM4 GPU with an AMD EPYC 7713 CPU, and an NVIDIA GH200 Grace Hopper Superchip. All code was compiled with NVIDIA CUDA Toolkit 12.9.86 and gcc 11.5.0. We compare the following methods with each other and with the native DGEMM and SGEMM from cuBLAS:

TF32GEMM: TF32 matrix multiplication using cublasGemmEx with CUBLAS_COMPUTE_32F_FAST_TF32.

BF16x9: SGEMM using cublasGemmEx with CUBLAS_COMPUTE_32F_EMULATED_16BFx9.

cuMpSGEMM: SGEMM using cuMpSGEMM in FP16TCEC_SCALING mode [8].

ozIMMU_EF-S: DGEMM using Ozaki I [17] with S slices.

OS II-fast-N: Ozaki II in fast mode with N moduli.

OS II-accu-N: Ozaki II in accurate mode with N moduli.

$A \in \mathbb{F}_b^{m \times k}$ and $B \in \mathbb{F}_b^{k \times n}$ for $b \in \{32, 64\}$ were generated as

$$a_{ij}, b_{ij} \leftarrow (\text{rand} - 0.5) \cdot \exp(\phi \cdot \text{randn}),$$

where $\phi \in \mathbb{F}_b$ controls the exponent distribution, $\text{rand} \in (0, 1] \subset \mathbb{F}_b$ is a uniformly distributed random number, and $\text{randn} \in \mathbb{F}_b$ is drawn from the standard normal distribution. Both rand and randn are generated using the cuRAND API with a fixed seed value. Empirically, the exponent distribution for matrix multiplication in HPL [3] is comparable to $\phi = 0.5$.

5.1 Accuracy

Figure 3 shows the accuracy of DGEMM emulation and SGEMM emulation on GH200. Similar results were obtained on A100 and RTX 5080. For $\phi = 0.5$, OS II-fast-14 yielded results with slightly lower accuracy than that of DGEMM, whereas OS II-accu-14 achieved accuracy comparable to that of DGEMM. OS II-fast-15 produced results with accuracy on par with DGEMM, while OS II-accu-15 provided slightly higher accuracy than DGEMM. These results imply that HPL can employ emulation with 14 or 15 moduli. For larger ϕ , the limiting accuracy of OS II-fast- N got worse as ϕ increased.

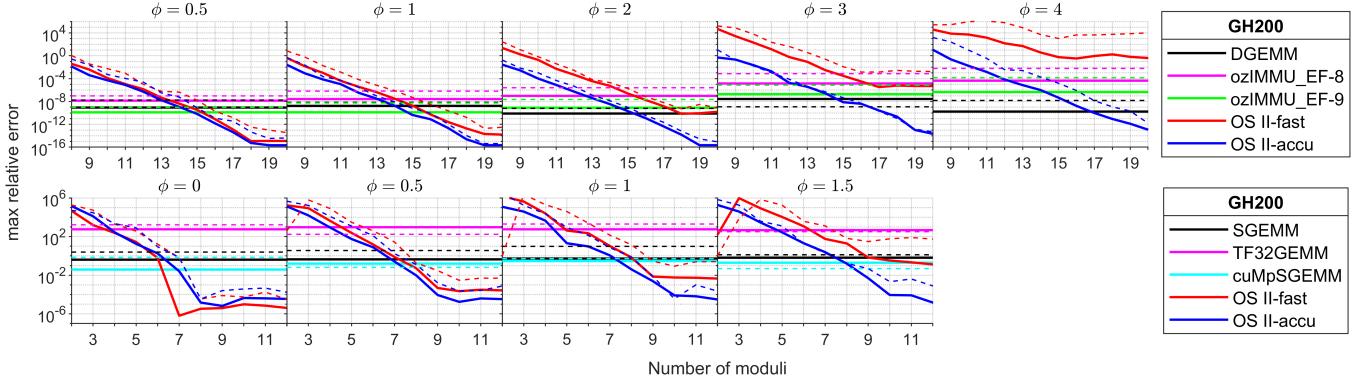


Figure 3: Accuracy of DGEMM (top) and SGEMM (bottom) emulation for $m = n = 1024$ on GH200. Solid lines represent results for $k = 1024$, and dashed lines for $k = 16384$.

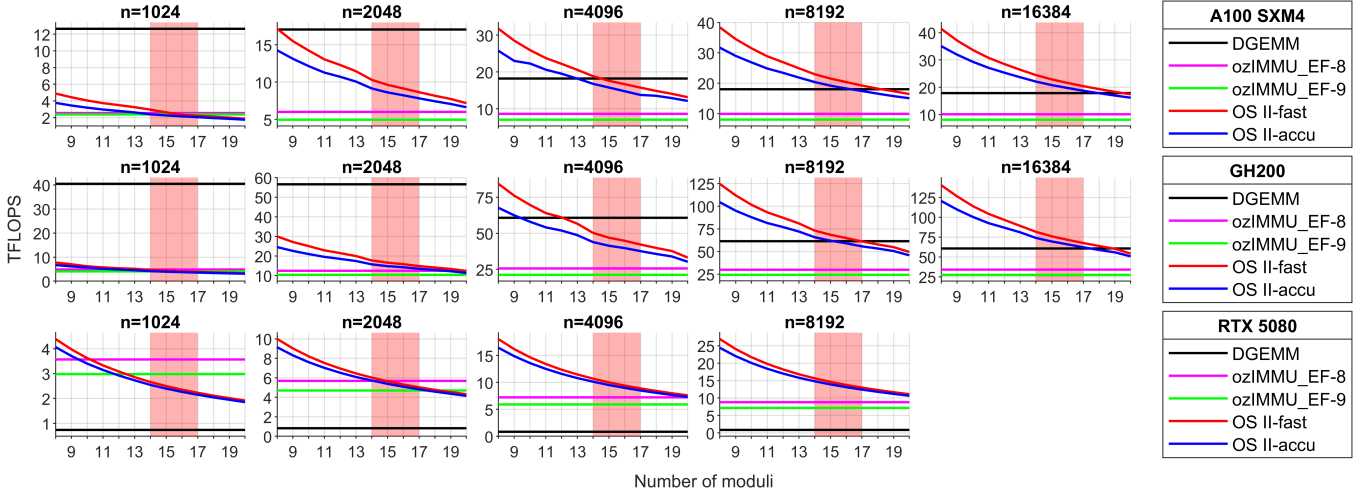


Figure 4: Throughput performance of DGEMM emulation on A100 (top), GH200 (middle), and RTX 5080 (bottom)

The overestimation of the upper bound of $\sum_{h=1}^k |a'_{ih}| |b'_{hj}|$ in (7) yields significantly small shift values μ and ν ; the truncation errors in the lines 2 and 3 of Algorithm 1 become huge in fast mode. In contrast, accurate mode estimates the upper bound by direct matrix multiplication. Therefore, OS II-accu- N exhibits smaller truncation errors compared to those of OS II-fast- N , allowing it to achieve sufficient accuracy with $N \leq 17$ even for $\phi = 4$.

On RTX 5080, SGEMM and BF16x9 exhibited equivalent accuracy. For $\phi \in \{0.5, 1, 1.5\}$, OS II-fast-2 yields $A' = O$ and $B' = O$ due to overestimation in (7). For $\phi \leq 1$, OS II-fast- N with $N \in \{7, 8\}$ returned results with SGEMM-level accuracy. For $\phi = 1.5$, the limiting accuracy of OS II-fast- N was around the accuracy of SGEMM; OS II-fast-9 achieved SGEMM-level accuracy for $k = 1024$. For $\phi \leq 1.5$, OS II-accu- N with $N \in \{6, 7, 8\}$ achieved accuracy comparable to that of SGEMM. In addition, OS II-fast- N with $N \in \{4, 5, 6, 7\}$ and OS II-accu- N with $N \in \{4, 5\}$ achieved TF32-level accuracy.

5.2 Throughput Performance

Figure 4 shows the throughput performance of DGEMM emulation. Note that the scales of the vertical axes are not uniform. According to Figure 3, $N \in \{14, 15, 16, 17\}$ yields a DGEMM-level accuracy. On RTX 5080, emulation methods were much faster than native DGEMM even for $n = 1024$ because FP64 is much slower than INT8 Tensor Cores, as shown in Figure 1. OS II-fast-14 and OS II-accu-14 achieved 18.5 $\times$ and 17.5 $\times$ speedups, respectively, compared to DGEMM for $n = 8192$. For DGEMM-level results for $n \leq 2048$, ozIMMU_EF- S was faster than OS II-fast- N and OS II-accu- N except for OS II-fast-14 for $n = 2048$ because computation in FP64 is the majority of Ozaki scheme II for small problems. For sufficiently large n , OS II-fast- N and OS II-accu- N were faster than ozIMMU_EF- S . On A100 and GH200, FP64 is not as slow as RTX 5080; therefore, OS II-fast- N and OS II-accu- N were faster than or comparable to ozIMMU_EF- S even for $n \leq 2048$. For $n \geq 8192$, OS II-fast- N and OS II-accu- N outperformed DGEMM because the overhead of computation in FP64 becomes relatively small. For $n = 16384$,

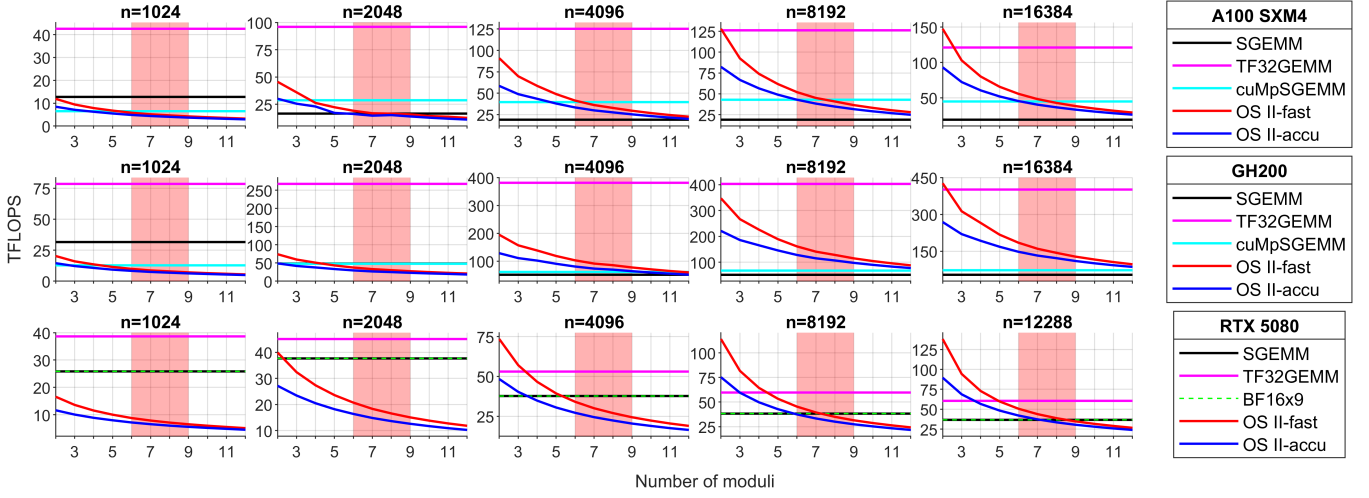


Figure 5: Throughput performance of SGEMM emulation on A100 (top), GH200 (middle), and RTX 5080 (bottom)

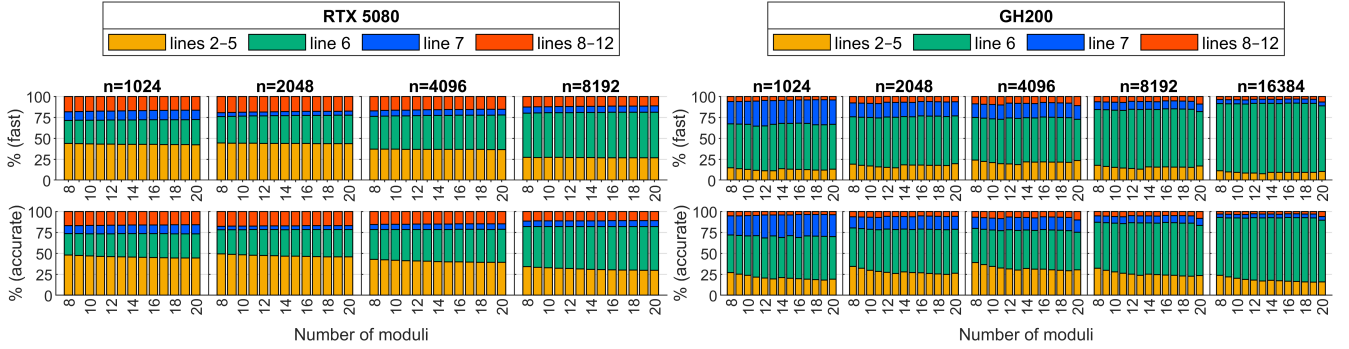


Figure 6: Time breakdown of DGEMM emulation in fast (top) and accurate (bottom) modes on RTX 5080 (left) and GH200 (right)

OS II-fast-14 achieved 24.5 TFLOPS on A100 and 81.6 TFLOPS on GH200, making it approximately 1.4× faster than DGEMM.

Figure 5 shows the throughput performance of SGEMM emulation. cuMpSGEMM was made from scratch using the NVIDIA WMMA API (Tensor Core device API) and optimized for A100. While it was not functional on RTX 5080, it outperformed SGEMM on GH200. The lines for SGEMM and BF16x9 overlap. For SGEMM-level results on RTX 5080, OS II-fast- N with $N \in \{6, 7, 8\}$ was faster than SGEMM and BF16x9 for $n = 12288$. On A100, as described above, cuMpSGEMM was optimized; however, OS II-fast-6 and 7 were faster than or comparable to cuMpSGEMM for $n \geq 4096$. For $n = 16384$ on GH200, OS II-accurate- N with $N \in \{6, 7, 8\}$ and OS II-fast- N with $N \in \{7, 8, 9\}$ achieved 122–148 TFLOPS and 128–160 TFLOPS, respectively, indicating that Ozaki scheme II achieved a 2.3–3.0× speedup compared to SGEMM. In addition, Ozaki scheme II demonstrated performance between those of SGEMM and TF32GEMM. As shown in Figure 3, Ozaki scheme II achieved accuracy between those of SGEMM and TF32GEMM, indicating that it can serve as an intermediate-precision approach between FP32 and TF32. Additional throughput results on an NVIDIA B200 Blackwell GPU are presented in Appendix A.

5.3 Time Breakdown

Figures 6 and 7 show the time breakdown of emulation of DGEMM and SGEMM on RTX 5080 and GH200 for $m = n = k$. The results for A100 were similar to those for GH200. Legend labels in the figures correspond to the respective lines in Algorithm 1. The conversion of input matrices in accurate mode includes matrix multiplication and accounts more computation time than that of fast mode. For DGEMM emulation on RTX 5080, due to low-performance FP64, even for $n = 8192$, non-matrix multiplication components accounted for around 50% of the entire computation time. For SGEMM emulation on RTX 5080, FP32 is 64× faster than FP64; thus, the conversion of input matrices is accelerated compared to that of DGEMM emulation. On A100 and GH200, for sufficiently large n , matrix multiplication is the major computation. As n increases, computations except for matrix multiplication gradually become negligible, suggesting that for $n \geq 16384$, Ozaki scheme II can be performed even more efficiently.

5.4 Power Efficiency

Figures 8 and 9 show the power efficiency (GFLOPS/watt) of emulation for $m = n = k$ on RTX 5080 and GH200. Similar trends

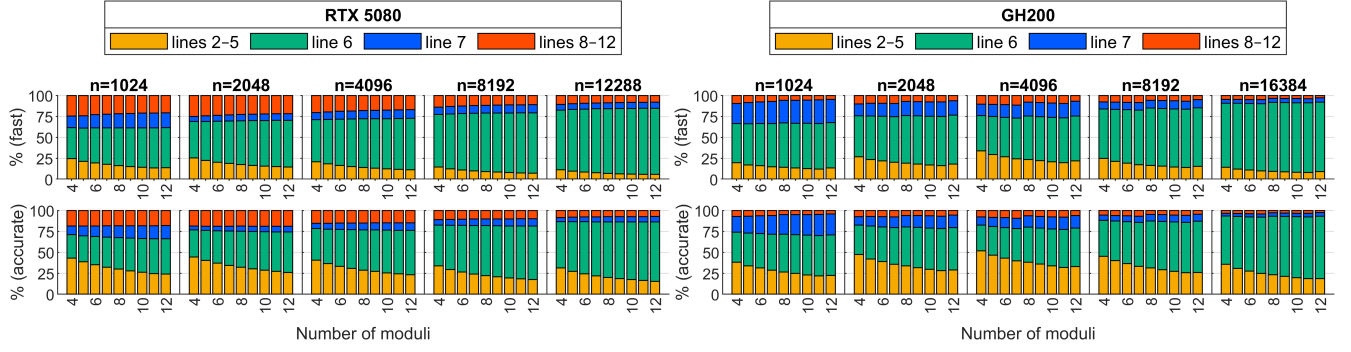


Figure 7: Time breakdown of SGEMM emulation in fast (top) and accurate (bottom) modes on RTX 5080 (left) and GH200 (right)

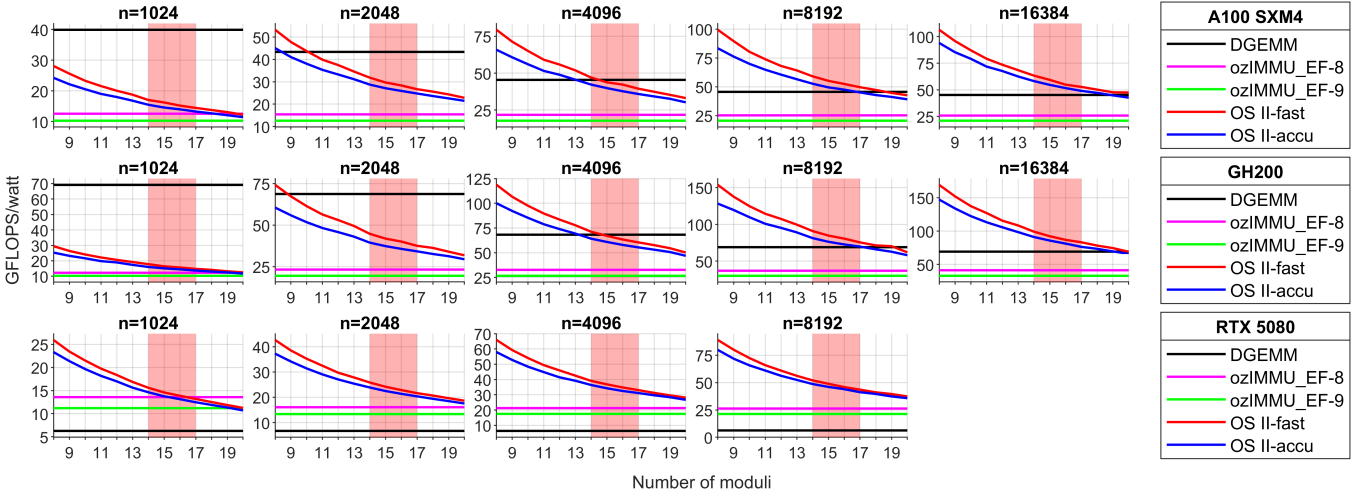


Figure 8: Power efficiency of DGEMM emulation on A100 (top), GH200 (middle), and RTX 5080 (bottom)

were observed on A100 as on GH200. The power efficiency exhibits trends similar to those of throughput performance, as shown in Figures 4 and 5. However, for smaller problem sizes, the results of Ozaki scheme II reached those of existing emulation, DGEMM, and SGEMM. This can be attributed to the relatively high power efficiency of INT8 GEMM even for moderately sized problems. For instance, the performance ratio between INT8 GEMM and SGEMM at $n = 1024$ was 5.3 $\times$, while the power efficiency ratio was as high as 13.3 $\times$ on RTX 5080. On GH200, OS II-fast- N and OS II-accu- N achieved 20%–43% and 10%–32% improvements, respectively, compared to DGEMM for $N \in \{14, 15, 16, 17\}$ and $n = 16384$. OS II-fast- N with $N \in \{7, 8, 9\}$ and OS II-accu- N with $N \in \{6, 7, 8\}$ achieved 103%–154% and 96%–137% improvements, respectively, compared to SGEMM for $n = 16384$.

6 Conclusion

This paper presented DGEMM and SGEMM emulation based on Ozaki scheme II. In many cases, applications employing DGEMM do not require the full precision of FP64 (see, e.g., [2]). Ozaki scheme I and II can deliver the required accuracy at a reasonable speed. Numerical experiments compared native GEMM routines and the

proposed emulation in terms of accuracy, throughput performance, and power efficiency. For sufficiently large problems, the proposed emulation outperformed native DGEMM, native SGEMM, and existing emulation methods in both performance and power efficiency. In addition, Ozaki scheme II can serve as an intermediate-precision approach between FP32 and TF32. It can also be extended to matrix multiplication using arbitrary combinations of floating-point formats, including both homogeneous (e.g., double-double) and heterogeneous (e.g., FP16 and FP32) types. The proposed methods help bridge the gap between low-precision computing units optimized for AI and numerical computations that requires high precision.

Acknowledgments

We appreciate the helpful comments on cuMPSGEMM from Dr. Hiroyuki Ootomo. We thank Patrick Gutsche, Prajval Kumar, and Dr. William Dawson for their assistance with debugging the code. We also thank Dr. Qianxiang Ma and Prof. Rio Yokota for their cooperation in conducting the experiments on the B200. This study was supported by Japan Society for the Promotion of Science Grant-in-Aid Numbers 25H01109 (Scientific Research A), 23K28100, 25K03126 (Scientific Research B), and 24K23874 (Research Activity Start-up).

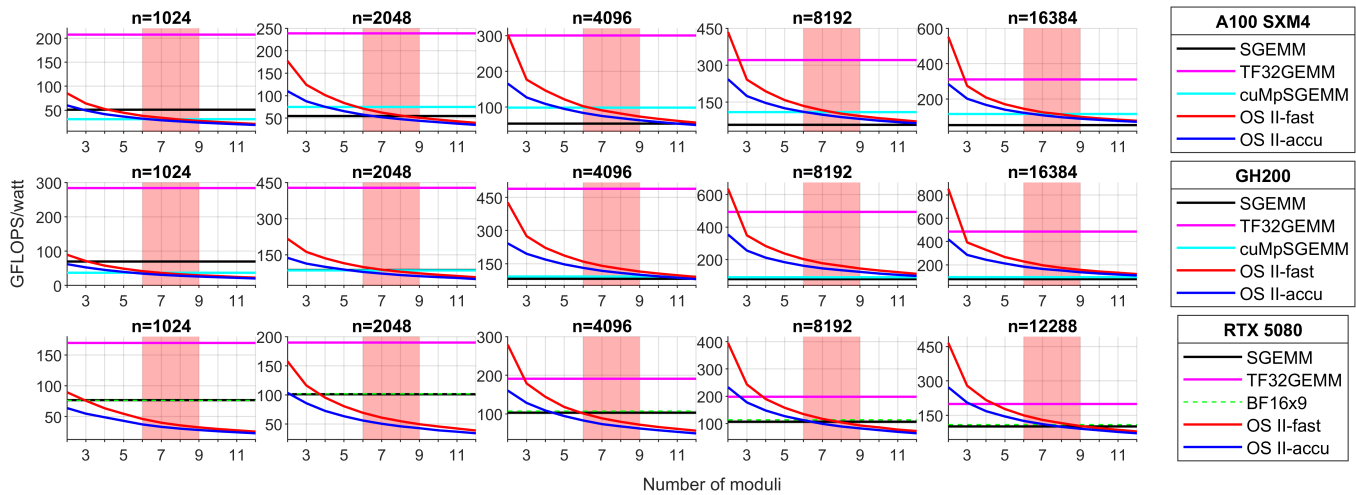


Figure 9: Power efficiency of SGEMM emulation on A100 (top), GH200 (middle), and RTX 5080 (bottom)

References

- [1] AMD Inc. 2025. *AMD matrix cores*. <https://rocm.blogs.amd.com/software-tools-optimization/matrix-cores/README.html> retrieved 21 March, 2025.
- [2] William Dawson, Katsuhisa Ozaki, Jens Domke, and Takahito Nakajima. 2024. Reducing Numerical Precision Requirements in Quantum Chemistry Calculations. *Journal of Chemical Theory and Computation* 20, 24 (2024), 10826–10837. doi:10.1021/acs.jctc.4c00938
- [3] Jack J Dongarra, Piotr Luszczek, and Antoine Petit. 2003. The LINPACK benchmark: past, present and future. *Concurrency and Computation: practice and experience* 15, 9 (2003), 803–820. doi:10.1002/cpe.728
- [4] Google LLC. 2025. *Cloud Tensor Processing Unit*. <https://cloud.google.com/tpu/docs/intro-to-tpu> retrieved 21 March, 2025.
- [5] Babak Hejazi, Mark Wolf, Roman Dubtsov, and Becca Zandstein. 2025. Boosting Matrix Multiplication Speed and Flexibility with NVIDIA cuBLAS 12.9. <https://developer.nvidia.com/blog/boosting-matrix-multiplication-speed-and-flexibility-with-nvidia-cublas-12-9/>
- [6] Greg Henry, Ping Tak Peter Tang, and Alexander Heinecke. 2019. Leveraging the bfloat16 Artificial Intelligence Datatype For Higher-Precision Computations. arXiv:1904.06376 [cs.MS]
- [7] NVIDIA Corporation. 2025. *NVIDIA Tensor Cores*. <https://www.nvidia.com/en-us/data-center/tensor-cores/> retrieved 25 July, 2025.
- [8] Hiroyuki Ootomo. 2023. *cuMpSGEMM - CUDA Mutable-precision SGEMM*. <https://github.com/enp1s0/cuMpSGEMM>
- [9] Hiroyuki Ootomo. 2024. *ozIMMU - DGEMM on Int8 Tensor Core*. <https://github.com/enp1s0/ozIMMU>
- [10] Hiroyuki Ootomo, Hidetaka Manabe, Kenji Harada, and Rio Yokota. 2023. Quantum Circuit Simulation by SGEMM Emulation on Tensor Cores and Automatic Precision Selection. In *High Performance Computing*. Springer Nature Switzerland, Cham, 259–276. doi:10.1007/978-3-031-32041-5_14
- [11] Hiroyuki Ootomo, Katsuhisa Ozaki, and Rio Yokota. 2024. DGEMM on integer matrix multiplication unit. *The International Journal of High Performance Computing Applications* 38, 4 (2024), 297–313. doi:10.1177/10943420241239588
- [12] Hiroyuki Ootomo and Rio Yokota. 2022. Recovering single precision accuracy from Tensor Cores while surpassing the FP32 theoretical peak performance. *The International Journal of High Performance Computing Applications* 36, 4 (2022), 475–491. doi:10.1177/10943420221090256
- [13] Katsuhisa Ozaki, Takeshi Ogita, Shin'ichi Oishi, and Siegfried M. Rump. 2012. Error-free transformations of matrix multiplication by using fast routines of matrix multiplication and its applications. *Numerical Algorithms* 59, 1 (2012), 95–118. doi:10.1007/s11075-011-9478-1
- [14] Katsuhisa Ozaki, Takeshi Ogita, Shin'ichi Oishi, and Siegfried M Rump. 2013. Generalization of error-free transformation for matrix multiplication and its application. *Nonlinear Theory and Its Applications, IEICE* 4, 1 (2013), 2–11. doi:10.1587/nolta.4.2
- [15] Katsuhisa Ozaki, Yuki Uchino, and Toshiyuki Imamura. 2025. Ozaki Scheme II: A GEMM-oriented emulation of floating-point matrix multiplication using an integer modular technique. arXiv:2504.08009 [cs.MS]
- [16] Ramesh Perumal, Nikitha Chinthalapani, and Rohit D'Souza. 2024. *Heterogeneous AI Powerhouse: Unveiling the Hardware and Software Foundation of Intel® Core™*
- [17] Yuki Uchino. 2024. *Accelerator for ozIMMU*. https://github.com/RIKEN-RCSS/accelerator_for_ozIMMU
- [18] Yuki Uchino. 2025. *GEMMul8: GEMM emulation using int8 matrix engines based on the Ozaki scheme II*. <https://github.com/RIKEN-RCSS/GEMMul8>
- [19] Yuki Uchino, Katsuhisa Ozaki, and Toshiyuki Imamura. 2025. Performance enhancement of the Ozaki Scheme on integer matrix multiplication unit. *The International Journal of High Performance Computing Applications* 39, 3 (2025), 462–476. doi:10.1177/10943420241313064

A Supplementary Experiment

Table 1 shows the throughput performance of matrix multiplication on an NVIDIA B200 Blackwell GPU.

Table 1: Throughput performance (in TFLOPS/TOPS) on an NVIDIA B200 Blackwell GPU with an Intel Xeon Platinum 8570 CPU, CUDA Toolkit 12.8.93, and g++ 13.3.0, for $m = k = n$.

methods \ n		1024	2048	4096	8192	16384
DGEMM	OS II-fast-14	3.72	21.3	71.4	119	123
	OS II-fast-17	3.04	18.0	59.0	99.1	98.9
	OS II-accur-14	3.36	19.9	63.3	107	110
	OS II-accur-17	2.85	17.0	54.1	90.2	92.5
SGEMM	OS II-fast-7	7.04	41.1	134	233	259
	OS II-fast-9	5.72	34.1	115	193	197
	OS II-accur-6	6.85	38.2	120	216	240
	OS II-accur-8	5.59	32.0	104	182	194
cuBLAS	DGEMM	24.3	34.1	35.1	35.7	36.0
	SGEMM	39.1	54.3	63.3	66.5	67.2
	TF32GEMM	98.9	433	734	726	702
	INT8GEMM	101	723	2362	3652	3238